

Student Performance Enhancement Prediction System

This document explains how the Python AI service works, what models it uses, and how data moves through the system step by step.

1. Purpose of the service

The Python service is the backend AI layer for the assessment platform. It accepts:

- exam/test performance data
- interview feedback text

It then produces:

- weak topics
- strong topics
- topic-level performance metrics
- communication and confidence insights
- suggested improvement areas
- personalized recommendations

The service is implemented with:

- FastAPI for the HTTP API
- TensorFlow / Keras for prediction models
- Pydantic for request and response validation
- NumPy for numeric feature handling

2. Main files and their roles

[app/main.py](../app/main.py)

This is the API entry point.

Main responsibilities:

- creates the FastAPI app
- enables CORS for the frontend
- loads trained models on startup
- exposes:

- `GET /health`

- `POST /analyze-performance`

Important functions:

- `bootstrap\_models()`
- `health()`
- `analyze\_performance(payload)`

[app/model.py](../app/model.py)

This file contains the TensorFlow model logic.

Main responsibilities:

- defines both AI models
- trains models from JSON training data
- saves trained artifacts
- loads trained models during inference
- predicts:

- topic weakness

- interview feedback scores

Important functions:

- `build\_topic\_weakness\_model(input\_dim, output\_dim)`
- `build\_feedback\_model(max\_tokens=1500, sequence\_length=120)`
- `train\_models(epochs=25)`
- `ensure\_models\_trained()`
- `predict\_topic\_weakness(test\_data)`
- `analyze\_feedback(feedback\_items)`

[app/preprocess.py](../app/preprocess.py)

This file converts raw request data into model-ready features.

Important functions:

- `load\_topics()`
- `aggregate\_test\_records(test\_data, weak\_threshold=0.6)`
- `build\_topic\_feature\_vector(aggregated\_topics, topic\_order)`
- `combine\_feedback\_text(feedback\_items)`

[app/utils.py](../app/utils.py)

This file turns raw model outputs into user-friendly insights and recommendations.

Important functions:

- ``clamp_score(value)``
- ``extract_feedback_highlights(tokens, limit=6)``
- ``build_improvement_areas(...)``
- ``build_recommendations(...)``
- ``build_communication_insights(...)``
- ``extract_tagged_items(feedback_texts, tag)``

[app/schemas.py](../app/schemas.py)

This file defines the API request and response formats using Pydantic.

Important models:

- ``TestRecord``
- ``InterviewFeedbackRecord``
- ``StudentPerformanceRequest``
- ``TopicMetric``
- ``FeedbackInsights``
- ``StudentPerformanceResponse``

[train.py](../train.py)

This is the training entry point.

It simply calls:

- ``train_models()``

and prints the saved artifact paths.

3. High-level architecture

The service has two prediction paths:

Model 1: Topic Weakness Predictor

Purpose:

- analyze test accuracy topic by topic
- estimate weakness probability for each topic

Input:

- numeric topic-wise accuracy vector

Output:

- weakness probability per topic

Architecture:

- Dense(32, relu)
- Dropout(0.15)
- Dense(16, relu)
- Dense(output_dim, sigmoid)

Why sigmoid?

- each topic gets an independent weakness probability between 0 and 1

Model 2: Feedback Analyzer

Purpose:

- analyze interview feedback text
- estimate:

- communication score
- confidence score
- technical skill score

Input:

- combined interview feedback text

Output:

- 3 continuous scores in the range 0 to 1, later scaled to 0 to 100

Architecture:

- `TextVectorization`

- ``Embedding(max_tokens, 32)``
- ``Bidirectional(LSTM(24))``
- ``Dense(24, relu)``
- ``Dense(3, sigmoid)``

4. Training pipeline step by step

Training starts from `[train.py](../train.py)`, which calls ``train_models()`` in `[app/model.py](../app/model.py)`.

Step 1: Load training datasets

``train_models()`` reads:

- ``data/performance_training.json``
- ``data/feedback_training.json``

``performance_training.json`` contains topic-level numeric training samples.

Typical fields:

- ``student_id``
- ``topic``
- ``accuracy``
- ``weak_label``

``feedback_training.json`` contains interview text with labeled scores.

Typical fields:

- ``text``
- ``communication_score``
- ``confidence_score``
- ``technical_skill_score``

Step 2: Build the topic vocabulary

The code extracts all unique topics from performance training data:

- ``topics = sorted({row["topic"] for row in performance_rows})``

That ordered topic list is saved to:

- ``models/topics.json``

This file is important because inference must use the exact same topic order as training.

Step 3: Prepare topic weakness training data

The code groups performance rows by student:

- ``grouped_rows``
- ``grouped_labels``

For each student:

- the feature vector stores topic accuracies
- the label vector stores weakness labels

These become:

- ``x_topic``: input matrix
- ``y_topic``: target matrix

Each row corresponds to one student, and each column corresponds to one topic.

Step 4: Train the topic weakness model

``build_topic_weakness_model()`` creates the dense neural network.

Then:

- ``model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])``
- ``model.fit(x_topic, y_topic, epochs=epochs, verbose=0)``

Finally the trained model is saved to:

- ``models/topic_weakness_model.keras``

Step 5: Prepare feedback training data

The feedback text column is converted to a TensorFlow string tensor:

- ``feedback_texts = tf.convert_to_tensor([...], dtype=tf.string)``

The three score targets are normalized to 0 to 1 by dividing by 100:

- communication

- confidence
- technical skill

These become:

- ``feedback_targets``

Step 6: Adapt the text vectorizer

The feedback model contains ``TextVectorization`` as its first layer.

Before training, the vectorizer is adapted to the training corpus:

- ``vectorizer.adapt(feedback_texts)``

This learns the service's internal vocabulary from the feedback data.

Step 7: Train the feedback model

The model is compiled with:

- optimizer: ``adam``
- loss: ``mse``
- metric: ``mae``

Then trained with:

- ``feedback_model.fit(feedback_texts, feedback_targets, epochs=epochs, verbose=0)``

Finally it is saved to:

- ``models/feedback_analyzer_model.keras``

5. Startup flow

When the FastAPI service starts, `[app/main.py](../app/main.py)` runs:

- ``bootstrap_models()``

That calls:

- ``ensure_models_trained()``

``ensure_models_trained()`` checks whether these three artifacts exist:

- ``models/topic_weakness_model.keras``
- ``models/feedback_analyzer_model.keras``
- ``models/topics.json``

If they are missing, the service trains them automatically.

So the API can bootstrap itself on first run.

6. Request flow for ``POST /analyze-performance``

The main inference path is the ``analyze_performance(payload)`` function in `[app/main.py](../app/main.py)`.

The request body is validated by ``StudentPerformanceRequest``.

Main fields:

- ``student_id``
- ``student_name``
- ``analysis_mode``
- ``test_data``
- ``interview_feedback``

``analysis_mode`` supports three modes:

- ``test``
- ``interview``
- ``combined``

7. Step-by-step data flow during inference

Mode A: Test mode

Used when:

- only test data should drive the analysis

Flow:

1. Request arrives with ``analysis_mode = "test"``.
2. ``predict_topic_weakness(payload.test_data)`` is called.
3. Test records are aggregated by ``(subject, topic)`` using ``aggregate_test_records()``.
4. Accuracy is computed for each topic.
5. ``build_topic_feature_vector()`` converts aggregated topics into a fixed-size vector matching ``topics.json``.

6. The saved topic weakness model predicts weakness probabilities.
7. The service builds topic metrics:
 - subject
 - topic
 - attempts
 - score
 - total marks
 - accuracy
 - weakness probability
 - is_weak
8. Weak topics and strong topics are extracted from those metrics.
9. Since this mode ignores interview text, feedback scores are set to zeros.
10. The response is built.

Mode B: Interview mode

Used when:

- only interview feedback should drive the analysis

Flow:

1. Request arrives with ``analysis_mode = "interview"`.`
2. ``analyze_feedback(payload.interview_feedback)`` is called.
3. ``combine_feedback_text()`` joins all feedback snippets into one text document.
4. The feedback model predicts:
 - communication score
 - confidence score
 - technical skill score
5. The text is also tokenized in a lightweight way for keyword highlights.
6. ``extract_feedback_highlights()`` returns the most frequent informative words.
7. ``extract_tagged_items()`` scans structured feedback blocks such as:
 - ``Topics To Improve:``
 - ``Subjects To Improve:``
 - ``Strengths:``
8. Those tagged sections are used to produce:
 - weak topics
 - strong topics
 - improvement areas

9. Since no test data is used, `topic\_metrics` is empty.

10. The response is built.

Mode C: Combined mode

Used when:

- both test data and interview feedback should be analyzed together

Flow:

1. Both prediction branches run:

- `predict\_topic\_weakness(...)`
- `analyze\_feedback(...)`

2. Weak topics from exam analysis are collected.

3. Weak topics from interview tagged feedback are collected.

4. Both sets are merged and deduplicated.

5. Strong topics are also merged and deduplicated.

6. Overall score is computed as the average of available test and interview scores.

7. Improvement areas and recommendations are built from both sources.

8. The combined response is returned.

8. Detailed explanation of the most important functions

`aggregate\_test\_records()`

Location:

- [app/preprocess.py](../app/preprocess.py)

What it does:

- groups raw test entries by `(subject, topic)`
- sums earned marks and total marks
- counts attempts
- computes `accuracy = score / total\_marks`
- marks a topic as weak if accuracy is below `weak\_threshold` (default 0.6)

This is the bridge from raw question-level data to topic-level analytics.

`build\_topic\_feature\_vector()`

Location:

- [app/preprocess.py](../app/preprocess.py)

What it does:

- takes the aggregated topic accuracy data
- matches it to the canonical topic order in `topics.json`
- fills missing topics with `0.0`
- returns a fixed-size NumPy vector

This step is critical because neural networks require a stable input dimension.

`predict\_topic\_weakness()`

Location:

- [app/model.py](../app/model.py)

What it does:

1. ensures models exist
2. loads `topics.json`
3. loads the saved weakness model
4. aggregates incoming test data
5. builds the feature vector
6. predicts per-topic weakness probabilities
7. enriches each topic row with:
 - `weakness\_probability`
 - final `is\_weak`

Important rule:

- a topic is weak if either:
 - its actual accuracy is below threshold
 - or model probability is at least 0.5

`analyze\_feedback()`

Location:

- [app/model.py](../app/model.py)

What it does:

1. ensures models exist
2. loads the saved feedback model
3. joins all interview feedback into one text string
4. runs the TensorFlow model to get 3 predicted scores
5. scales them from 0 to 100
6. clamps them using ``clamp_score()``
7. extracts top keyword highlights from the text

Returned fields:

- ``communication_score``
- ``confidence_score``
- ``technical_skill_score``
- ``highlights``

``extract_tagged_items()``

Location:

- `[app/utils.py](../app/utils.py)`

What it does:

- extracts structured list-like values from feedback text

Example input:

- ``Topics To Improve: Database indexes, Database views``

Example output:

- ``["Database indexes", "Database views"]``

This function is very important for the current interview integration because it lets the service use structured Gemini feedback directly instead of re-reading the whole raw interview transcript.

``build_improvement_areas()``

Location:

- `[app/utils.py](../app/utils.py)`

What it does:

- converts weak topics and low soft-skill scores into human-readable focus areas

Examples:

- `Revise Arrays`
- `Improve communication clarity`
- `Build interview confidence`

`build\_recommendations()`

Location:

- [app/utils.py](../app/utils.py)

What it does:

- turns the analysis into concrete action steps

Examples:

- practice topic revision
- record mock interviews
- improve concise speaking

9. How scores are computed

Test score

Computed as:

- average accuracy across aggregated topic metrics
- then multiplied by 100

Interview score

Computed as:

- average of:
 - communication score
 - confidence score

- technical skill score

Overall score

Depends on the mode:

- `test`: overall score = test score
- `interview`: overall score = interview score
- `combined`: overall score = average of available test and interview scores

10. Response construction

The API returns `StudentPerformanceResponse`.

Main sections:

- `weak\_topics`
- `strong\_topics`
- `suggested\_improvement\_areas`
- `communication\_insights`
- `feedback\_scores`
- `topic\_metrics`
- `overall\_performance\_score`
- `generated\_recommendations`

This response is already shaped for:

- Next.js frontend rendering
- Postman testing
- future reporting/export features

11. Example end-to-end data flow

Suppose the frontend sends:

- test records for Arrays, SQL, and Networking
- interview feedback containing:
 - `Topics To Improve: Database indexes, Database views`
 - `Subjects To Improve: Database design`
 - `Strengths: SQL joins, normalization`

Then the service does this:

1. validates request with Pydantic
2. aggregates test records by topic
3. predicts topic weakness probabilities with the dense model
4. combines interview text into one document
5. predicts communication, confidence, and technical scores with the LSTM model
6. extracts structured interview topics and subjects from tagged fields
7. merges exam and interview weakness signals in combined mode
8. generates readable insights and recommendations
9. returns one final JSON response

12. Why this design works well

Strengths of the design:

- clear separation between API, preprocessing, modeling, and utility logic
- automatic model bootstrapping on startup
- separate numeric and NLP pipelines
- structured schemas make frontend integration safer
- combined mode can fuse exam and interview signals
- recommendations are deterministic and easy to understand

13. Current limitations

Important practical limitations in the current implementation:

- the models are relatively small and depend on the quality of training JSON data
- the topic weakness model treats missing topics as zero-accuracy inputs
- there is no persistent experiment tracking or model versioning
- the service currently loads models from disk during inference calls instead of long-lived cached instances
- CORS is configured mainly for local development
- production deployment needs a hosted Python runtime and public API URL

14. Summary

In short, the Python AI service works like this:

1. FastAPI receives test and/or interview data.
2. Pydantic validates the request.
3. Preprocessing converts raw records into model-ready features.
4. TensorFlow model 1 predicts topic weakness from exam accuracy vectors.

5. TensorFlow model 2 predicts communication, confidence, and technical skill from interview feedback text.
6. Utility functions turn model outputs into weak topics, insights, improvement areas, and recommendations.
7. FastAPI returns a structured JSON response to the frontend.

That is the complete flow from raw student data to AI-guided learning advice.